

Syntax Analysis (Part 2) - Top-Down Parsing

1. Introduction to Top-Down Parsing

What is Top-Down Parsing?

Top-down parsing is a strategy where the parser attempts to construct a parse tree **from the root (start symbol) down to the leaves (input terminals)**.

Key Characteristics:

1. **Direction of Derivation:** Finds the **leftmost derivation** of the input string
2. **Parse Tree Construction:** Built from top (root) to bottom (leaves)
3. **Prediction-based:** The parser tries to predict which production rule to use based on the current input

Core Idea

The parser:

- Maintains a stack of grammar symbols
- Tries to match non-terminals on the stack to terminals in the input
- Chooses production rules based on the current input symbol
- Works from the start symbol toward the actual input

Example Concept

Grammar:

$S \rightarrow aB$

$B \rightarrow b \mid c$

Input: "ab"

Top-Down Process:

1. Start with S
2. Predict: $S \rightarrow aB$ (because input starts with 'a')
3. Match 'a' with input
4. Predict: $B \rightarrow b$ (because next input is 'b')
5. Match 'b' with input
6. Success!

2. Types of Top-Down Parsers

There are two main types of top-down parsers, each with different characteristics:

Type 1: Recursive Descent Parser

Description:

- A collection of **mutually recursive functions**. Where each function corresponds to one non-terminal in the grammar
- Functions directly implement production rules

Mechanism:

- Each non-terminal has a corresponding parsing function
- Functions call each other recursively
- May use **backtracking**: try one rule, if it fails, undo and try another

Advantages:

- Easy to understand and implement
- Directly corresponds to grammar structure
- Good for hand-written parsers

Drawbacks:

- Inefficient due to backtracking
- Can lead to exponential time complexity for some grammars
- May try many wrong paths before finding correct one
- Difficult to implement for complex grammars

Backtracking Example:

Grammar: $S \rightarrow aB \mid aC$

$B \rightarrow b$

$C \rightarrow c$

Input: "ac"

Process with backtracking:

1. Try $S \rightarrow aB$
2. Match 'a' ✓

3. Try $B \rightarrow b$
4. Next input is 'c', not 'b' ✗
5. BACKTRACK: Undo matching 'a'
6. Try $S \rightarrow aC$
7. Match 'a' ✓
8. Try $C \rightarrow c$
9. Match 'c' ✓
10. Success!

Type 2: Predictive Parser (LL(1))

Description:

- A special case of recursive descent that **does NOT require backtracking**
- Uses **lookahead** to deterministically choose the correct production
- Most commonly implemented as **LL(1) parser**

Mechanism:

- Uses a **stack** to store grammar symbols
- Uses a **parsing table** (built using FIRST and FOLLOW sets)
- Makes decisions with **1 symbol of lookahead**
- **Deterministic:** always knows which rule to apply

Advantages:

- **No backtracking:** efficient, linear time complexity $O(n)$
- Deterministic parsing
- Can be automatically generated from grammar
- Predictable performance

Drawbacks:

- **Requires grammar in special form** (LL(1) grammar)
- Not all grammars can be converted to LL(1)
- Must eliminate left recursion
- Must perform left factoring

Comparison Table

Feature	Recursive Descent	Predictive (LL(1))
Backtracking	May use it	No backtracking
Efficiency	Can be exponential	Always $O(n)$
Grammar Requirements	Any grammar	Must be LL(1)
Implementation	Recursive functions	Stack + Table
Determinism	Non-deterministic	Deterministic
Lookahead	May look ahead multiple symbols	Exactly 1 symbol
Automatic Generation	Difficult	Easy (from table)

3. LL(1) Parsing in Detail

What is LL(1)?

LL(1) parsing is a method of **non-recursive top-down parsing** that operates without backtracking. It is **deterministic**, meaning it can always predict the next production rule based on:

- The current non-terminal on the stack
- One symbol of lookahead from the input

The LL(1) Notation Explained

The name "**LL(1)**" describes three aspects of the parser:

L (First L): Left-to-right scanning

- The input is read from left to right
- Process one token at a time, moving forward

L (Second L): Leftmost derivation

- The parser produces a leftmost derivation
- Always expand the leftmost non-terminal first

1: 1 symbol of lookahead

- The parser looks at only one upcoming input symbol
- Uses this single symbol to make decisions

Why LL(1) is Important

1. **Efficient:** Linear time complexity $O(n)$
2. **Deterministic:** No guessing or backtracking
3. **Predictable:** Always knows what to do
4. **Automatic:** Can be generated from grammar rules
5. **Practical:** Used in many real compilers

4. The Four Core Components of LL(1) Parser

An LL(1) parser consists of four main parts working together:

1. **Input Buffer**
 - Stores the input string (tokens) one by one.
 - The input always ends with a special symbol **\$** to show the end.
2. **Stack**
 - Used to keep grammar symbols (terminals and non-terminals).
 - At the beginning, it contains **\$** at the bottom and the **start symbol S** on top:
[**\$ S**]
3. **Parsing Program**
 - Controls the parsing process.
 - It looks at:
 - the **top of the stack**
 - the **current input symbol**
 - Then decides what action to take.
4. **LL(1) Parsing Table (M)**
 - A table that tells the parser which rule to use.
 - **Rows** → Non-terminals
 - **Columns** → Terminals (including **\$**)
 - Each entry **M[A, a]** gives the production **A → α** to apply.

5. Requirements and Limitations

LL(1) parsing is **only possible** if the grammar satisfies strict conditions. These ensure the parsing table is **deterministic** (each cell has at most one production).

1. No Left Recursion

- The grammar must not have direct or indirect left recursion.
- Left recursion causes an infinite loop in LL(1) parsing.
- This problem is fixed by converting the grammar into right-recursive form.

2. No Left Factoring Conflict

- If a non-terminal has more than one production, their FIRST sets must be different.
- If FIRST sets overlap, the parser cannot decide which rule to use.
- This problem is fixed by left factoring the grammar.

6. FIRST and FOLLOW Sets

These sets are crucial for constructing the LL(1) parsing table.

FIRST Set

Definition: $\text{FIRST}(\alpha)$ is the set of terminals that can appear as the **first symbol** of any string derivable from α .

If α can derive ϵ , then $\epsilon \in \text{FIRST}(\alpha)$

Computing FIRST Sets

Rules:

1. $\text{FIRST}(a) = \{a\}$ where a is a terminal
 - A terminal's FIRST set is itself
2. If $X \rightarrow \epsilon$ is a production, then add ϵ to $\text{FIRST}(X)$
3. If $X \rightarrow Y_1 Y_2 \dots Y_k$:
 - Add $\text{FIRST}(Y_1) - \{\epsilon\}$ to $\text{FIRST}(X)$
 - If $\epsilon \in \text{FIRST}(Y_1)$, add $\text{FIRST}(Y_2) - \{\epsilon\}$
 - Continue until a Y_i that doesn't derive ϵ
 - If all Y 's can derive ϵ , add ϵ to $\text{FIRST}(X)$

FOLLOW Set

Definition: FOLLOW(A) is the set of terminals that can appear **immediately after** non-terminal A in some derivation.

\$ is always in FOLLOW(Start Symbol)

Computing FOLLOW Sets

Rules:

1. Place \$ in FOLLOW(S) where S is the start symbol

2. If $A \rightarrow \alpha B \beta$:

- Add $\text{FIRST}(\beta) - \{\epsilon\}$ to $\text{FOLLOW}(B)$
- If $\epsilon \in \text{FIRST}(\beta)$, add $\text{FOLLOW}(A)$ to $\text{FOLLOW}(B)$

3. If $A \rightarrow \alpha B$ (B is at the end):

- Add $\text{FOLLOW}(A)$ to $\text{FOLLOW}(B)$

Example: Computing FIRST and FOLLOW

Grammar:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \epsilon$

$F \rightarrow (E) \mid \text{id}$

Computing FIRST:

$\text{FIRST}(F) = \{ (, \text{id} \}$

$\text{FIRST}(T') = \{ *, \epsilon \}$

$\text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$

$\text{FIRST}(E') = \{ +, \epsilon \}$

$\text{FIRST}(E) = \text{FIRST}(T) = \{ (, \text{id} \}$

Computing FOLLOW:

$\text{FOLLOW}(E) = \{ \$,) \}$ // E is start, also in $F \rightarrow (E)$

$\text{FOLLOW}(E') = \text{FOLLOW}(E) = \{ \$,) \}$

$\text{FOLLOW}(T) = \text{FIRST}(E') \cup \text{FOLLOW}(E') = \{ +, \$,) \}$

$\text{FOLLOW}(T') = \text{FOLLOW}(T) = \{ +, \$,) \}$

$\text{FOLLOW}(F) = \text{FIRST}(T') \cup \text{FOLLOW}(T) = \{ *, +, \$,) \}$

7. Constructing the LL(1) Parsing Table

Algorithm

For each production $A \rightarrow \alpha$:

1. For each terminal a in $\text{FIRST}(\alpha)$:

- Add $A \rightarrow \alpha$ to $M[A, a]$

2. If $\epsilon \in \text{FIRST}(\alpha)$:

- For each terminal b in $\text{FOLLOW}(A)$:
 - Add $A \rightarrow \alpha$ to $M[A, b]$
- If $\$ \in \text{FOLLOW}(A)$:
 - Add $A \rightarrow \alpha$ to $M[A, \$]$

3. All undefined entries are errors

Example: Building Parsing Table

Grammar:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \epsilon$

$F \rightarrow (E) \mid \text{id}$

FIRST and FOLLOW (from previous):

$\text{FIRST}(E) = \{ (, \text{id} \}$ $\text{FOLLOW}(E) = \{ \$,) \}$

$\text{FIRST}(E') = \{ +, \epsilon \}$ $\text{FOLLOW}(E') = \{ \$,) \}$

$\text{FIRST}(T) = \{ (, \text{id} \}$ $\text{FOLLOW}(T) = \{ +, \$,) \}$

$\text{FIRST}(T') = \{ *, \epsilon \}$ $\text{FOLLOW}(T') = \{ +, \$,) \}$

$\text{FIRST}(F) = \{ (, \text{id} \}$ $\text{FOLLOW}(F) = \{ *, +, \$,) \}$

Constructing Table:

For $E \rightarrow T E'$:

- $\text{FIRST}(T E') = \{ (, \text{id} \}$
- $M[E, (] = E \rightarrow T E'$

- $M[E, id] = E \rightarrow T E'$

For $E' \rightarrow + T E'$:

- $FIRST(+ T E') = \{+\}$
- $M[E', +] = E' \rightarrow + T E'$

For $E' \rightarrow \epsilon$:

- $\epsilon \in FIRST(\epsilon)$
- $FOLLOW(E') = \{\$, \}$
- $M[E', \$] = E' \rightarrow \epsilon$
- $M[E',)] = E' \rightarrow \epsilon$